Installing Python

A small guide using uv

Why we use `uv` for this course

`uv` is a new (and very fast) Python tool written in Rust. It: - Installs Python for you (no manual downloads). - Creates isolated *virtual environments* (safe sandboxes per project). - Installs and updates packages quickly.

> **Note**
>
> WHAT is a virtual environment? Think of each project as its own coffee shop with its own supplies. One shop changing its menu does **not** affect the others. WHY it matters: You avoid random breakage when different projects need different versions of the same package.

Install `uv`

Choose the instructions for your operating system. For this, you need to work in your terminal.

> **Tip**
>
> Open the terminal in VS Code by selecting (Terminal → New Terminal).

macOS or Linux (Terminal)

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

If `curl` is missing:

```
wget -qO- https://astral.sh/uv/install.sh | sh
```

After installation: **close and reopen** your terminal (so your PATH updates).

Windows (PowerShell)

Open PowerShell and run:

```
powershell -ExecutionPolicy Bypass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

If you see a script execution warning, you can alternatively first run:

```
Set-ExecutionPolicy -ExecutionPolicy Bypass -Scope Process
```

Then re-run the install line.

Verify installation

Run (macOS / Linux / Windows):

```
uv --version
```

If you see a version number: great!

> ⚠ Warning
>
> If you get “command not found” or “‘uv’ is not recognized”:
>
> 1. Close and reopen the terminal (important).
> 2. On Windows: make sure you used PowerShell (not Command Prompt).
> 3. Still broken? Ask for help, no need of guessing the error.

Install Python

We want everyone on the **same** Python version for consistency. Thus, we’ll use Python 3.13 for the course this year.

Install (you only need to do this once):

```
uv python install 3.13
```

Check the installation:

```
uv run python --version
```

Expected output starts with:

```
Python 3.13.
```

Create your first project

Pick a folder where you keep course work. **If you do not have one, make sure to create one!** Open the course folder in your IDE and then run the following from the terminal:

```
uv init
```

`uv init` creates:

- `main.py` (starter script)
- `pyproject.toml` (project + dependencies config)

- `.python-version` (records the Python version of the project)
- `README.md` (you can jot notes here)
- a Git repository (`.git` folder and `.gitignore`), if Git is installed. You can ignore it for now.
- (A `.venv` folder and a `uv.lock` file appear the first time you run `uv run` or add a package.)

To make sure the project uses the course version, pin it once:

```
uv python pin 3.13
```

You do **not** need to edit any of these (except maybe `README.md` for your notes and `main.py` if you want to run something different).

Run the starter script

Inside the project folder **in your terminal**:

```
uv run python main.py
```

You should see a greeting with the name of your folder, for example for a folder called `programming-course`:

```
Hello from programming-course!
```

(If you want, you can open `main.py` and change the message, then re-run.)

What does that code mean?

```
def main():
    print("Hello from programming-course!")

if __name__ == "__main__":
    main()
```

- `def main():` defines a function (a reusable block of code).
- `print(...)` shows text in the terminal.
- The line `if __name__ == "__main__":` ensures this only auto-runs when the file is executed directly.

Don't worry about this yet, we'll gradually build up to it.

Add notebooks to your project

The tutorials are notebooks. Inside the project folder **in your terminal**, run the two commands from the introduction, one after the other:

```
uv add --dev ipykernel
```

```
uv add jupyter
```

The first command adds a kernel to the project, the second the Jupyter tools. Now notebooks can use the Python of your project.

Quick check before you continue:

- ⍰ You have a folder for the course
- ⍰ You ran `uv init` inside the folder
- ⍰ You can see a file called `pyproject.toml` in the folder
- ⍰ You ran `uv add --dev ipykernel` and `uv add jupyter`

Download a tutorial and open it

1. Open a tutorial on the website and select `Jupyter` on the right.
2. Save the file **to your course folder**.
3. Open it in VS Code and select "Open with Jupyter Notebook".

That was the hardest part. If something does not work, ask us.

Adding packages (later in the course)

If/when you need a package (example: `pandas`):

```
uv add pandas
```

If you added the wrong one:

```
uv remove pandas
```

If your `pyproject.toml` changed (e.g. you pulled code from someone else):

```
uv sync
```

> Tip
>
> If something seems "off", just close the terminal and reopen in the project folder. Fresh starts fix many early mistakes.

Updating `uv`

Occasionally:

```
uv self update
```

(If it ever errors, you can just reinstall using the same one-liner from earlier.)

Best practices for this course

- One folder for the course keeps everything tidy.
- Never install packages “globally” outside a project.
- Keep a short personal log in each project’s `README.md` (What did I do? What still confuses me?).
- Ask early for help, guessing usually takes much more time than asking.

You can always see available commands:

```
uv --help
```

Recap

You can now: 1. Install `uv`. 2. Create a project. 3. Run a script. 4. Add notebooks to your project and open a tutorial. 5. Add/remove/sync packages.

Now, you’re set to continue the course.